
Guía paso a paso — Gestor de Tareas (Task Manager)

Asignatura: Laboratorio Web 1 — Tecnicatura Universitaria en Desarrollo Web.

Profesor: Jorge Ceferino Valdez.

Público objetivo: Alumnos con introducción previa a HTML5, CSS3 y JavaScript vanilla que desean consolidar sus conocimientos mediante un proyecto práctico y progresivo.

Tecnologías: HTML5, CSS3 vanilla, JavaScript vanilla (sin frameworks ni librerías).

Enfoque pedagógico: Cada tarea se apoya en la anterior, permitiendo ver la evolución de un “sitio estático” a una “aplicación web con persistencia de datos”. No se introducen patrones de diseño formales; sí se muestran patrones de maquetación y, hacia el final, una refactorización guiada (extraer una función) razonada paso a paso.

Tabla de contenidos

1. Visión general del proyecto
 2. Cómo leer esta guía
 3. Tarea 1 — Página HTML básica
 4. Tarea 2 — Añadir interactividad
 5. Tarea 3 — Marcar tareas como completadas
 6. Tarea 4 — Botón de eliminar
 7. Tarea 5 — Contador de tareas
 8. Tarea 6 — División entre Activas y Completadas
 9. Tarea 7 — Persistencia y la función de renderizado
 10. Arquitectura final: decisiones clave
 11. Checklist de aprendizaje
-

1. Visión general del proyecto

Este ejercicio construye un gestor de tareas funcional en el navegador. Al finalizar las 7 tareas, la aplicación permite:

- Visualizar tareas predefinidas.
- Agregar nuevas tareas desde un campo de texto.
- Marcar/desmarcar tareas como completadas (con estilo visual diferenciado).
- Eliminar tareas individualmente.
- Ver contadores de tareas activas, completadas y total.
- Persistir la lista entre recargas de página usando `localStorage`.
- Borrar todas las tareas de un solo clic.

El proyecto se compone de tres archivos, uno por cada **capa** de una página web:

Archivo	Capa	Responsabilidad
index.html	Estructura	Qué elementos existen en la página
estilo.css	Presentación	Cómo se ven esos elementos
script.js	Conducta y datos	Qué hace la página y cómo gestiona la información

La idea central de la cursada es que cada tarea **agrega una capacidad** sin romper lo anterior. La aplicación es usable desde la Tarea 1; lo que cambia es cuánto puede hacer.

2. Cómo leer esta guía

Cada tarea sigue siempre el mismo esquema:

1. **Objetivo** — qué problema concreto vamos a resolver.
2. **Pasos** — el código, presentado en el orden en que conviene escribirlo.
3. **Decisiones de implementación** — el porqué de **cada línea**. Esta es la parte más importante: copiar código es fácil; entender por qué está escrito así es lo que se evalúa.

Una recomendación de método: **escribí el código vos mismo**, no lo copies y pegues. Al teclear cada línea te vas a hacer preguntas (“¿por qué const y no let?”, “¿por qué textContent y no innerHTML?”) y esta guía las responde justo al lado.

A partir de la Tarea 7 vas a ver una técnica nueva: **escribir primero el código “suelto” y luego reorganizarlo dentro de una función**. Eso es una *refactorización*. No es que la primera versión esté “mal”: es que, al verla repetirse, descubrimos por nosotros mismos la necesidad de encapsularla. Esa secuencia “primero el problema, después la solución” es deliberada.

3. Tarea 1 — Página HTML básica

Objetivo

Crear la estructura semántica del documento, listar tareas predefinidas y aplicar estilos básicos para que la página se vea ordenada y centrada.

Paso 1.1: Estructura HTML semántica

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Gestor de Tareas</title>
  <link rel="stylesheet" href="estilo.css">
</head>
<body>
  <h1>Gestor de tareas</h1>
  <ul id="task-list">
    <li>Comprar comida</li>
    <li>Pasear al perro</li>
    <li>Estudiar JavaScript</li>
    <li>Lavar la ropa</li>
    <li>Llamar a mamá</li>
  </ul>
  <script src="script.js"></script>
</body>
</html>
```

Decisiones de implementación, línea por línea:

- **<!DOCTYPE html>**: Le indica al navegador que use el modo estándar de HTML5. Sin esta línea, algunos navegadores entran en “quirks mode” (modo de compatibilidad antiguo) y el CSS se interpreta de forma inconsistente.
- **<html lang="es">**: El atributo lang declara el idioma del contenido. Mejora la accesibilidad (los lectores de pantalla eligen la voz correcta) y el comportamiento del navegador (corrector ortográfico, traducción automática).
- **<meta charset="UTF-8">**: Define la codificación de caracteres. Garantiza que tildes, eñes y el símbolo × que usaremos más adelante se rendericen correctamente. Va lo más arriba posible dentro del <head> para que el navegador sepa cómo decodificar el resto del archivo.
- **<title>**: Texto que aparece en la pestaña del navegador. No es decorativo: es lo que se ve en el historial y en los marcadores.
- **<link rel="stylesheet" href="estilo.css">**: Vincula la hoja de estilos **externa**. Mantener el CSS en su propio archivo separa la presentación de la estructura: podemos cambiar todos los colores sin tocar el HTML.
- **<h1>**: Encabezado de nivel 1. Cada página debería tener un único <h1> que describa su contenido principal. Es una señal semántica fuerte para buscadores y lectores de pantalla.
- **<ul id="task-list">**: Una lista **no ordenada** (ul) porque el orden de las tareas no implica jerarquía ni secuencia obligatoria. El atributo id="task-list" le da un identificador único; será el “punto de enganche” por el que JavaScript va a localizar y manipular la lista en las próximas tareas.
- ** con texto plano**: En esta etapa, cada tarea es solo texto dentro de un ítem de lista. Es suficiente. Más adelante necesitaremos más estructura interna (un para el texto, un <button> para borrar), pero no complicamos la base antes de necesitarlo.
- **<script src="script.js"> al final del <body>**: El navegador lee el HTML de arriba hacia abajo. Si el <script> estuviera en el <head>, se ejecutaría **antes** de que existan los elementos que quiere manipular. Colocándolo justo antes de </body> garantizamos que todo el DOM ya está construido cuando el script corre. (Por eso, más adelante, no necesitaremos envolver el código en DOMContentLoaded.)

Paso 1.2: CSS básico centrado

```
body {
  background-color: #f0f4f8;
  font-family: Arial, sans-serif;
  display: flex;
  flex-direction: column;
  align-items: center;
  margin: 0;
  padding: 20px;
}

h1 {
  margin-bottom: 10px;
}

ul#task-list {
  list-style: none;
  padding: 0;
  width: 100%;
  max-width: 600px;
}
```

Decisiones de implementación, línea por línea:

- **background-color: #f0f4f8**: Un gris azulado muy claro. Da un fondo limpio y profesional que no compite con el contenido. Usar texto oscuro sobre fondo claro maximiza la legibilidad.
- **font-family: Arial, sans-serif**: Pedimos Arial; si el sistema no la tiene, cae en cualquier sans-serif disponible. Esta “lista de respaldo” es la forma estándar de manejar fuentes en CSS.
- **display: flex**: Convierte al <body> en un **contenedor flex**. Es el mecanismo moderno para alinear y distribuir elementos sin recurrir a trucos antiguos (margin: auto, float).
- **flex-direction: column**: Apila los hijos del <body> uno debajo del otro (título, lista, etc.), en lugar de uno al lado del otro.
- **align-items: center**: En un contenedor en columna, alinea los hijos en el **eje horizontal**. Resultado: todo queda centrado en el ancho de la pantalla.
- **margin: 0**: Los navegadores aplican un margen por defecto al <body>. Lo eliminamos para tener control total y evitar una franja blanca alrededor del fondo.
- **padding: 20px**: Da un respiro entre el borde de la ventana y el contenido, para que nada quede pegado al margen.

- **h1 { margin-bottom: 10px }:** Acerca el título a la lista. Ajuste fino de ritmo visual.
- **ul#task-list:** El selector combina el tipo de elemento (ul) con su id. Es más específico que #task-list a secas y deja claro, al leer el CSS, que esta regla aplica a una lista.
- **list-style: none:** Elimina los puntos (bullets) de la lista. Vamos a darle a cada tarea un aspecto de “tarjeta”, y los bullets estorbarían.
- **padding: 0:** Las listas traen una sangría izquierda por defecto. La quitamos para que las tarjetas ocupen todo el ancho disponible.
- **width: 100% + max-width: 600px:** La lista intenta ocupar todo el ancho del contenedor, pero **nunca más de 600 px**. En un monitor grande, una línea de texto demasiado larga es incómoda de leer; este límite mantiene una medida cómoda.

4. Tarea 2 — Añadir interactividad

Objetivo

Permitir que el usuario agregue nuevas tareas dinámicamente desde la interfaz, sin tocar el código HTML.

Paso 2.1: Agregar controles al HTML

```
<form id="add-form" onsubmit="return false;">
  <input type="text" id="add-input" placeholder="Nueva tarea" autocomplete="off" />
  <button type="button" id="add-btn">Añadir tarea</button>
</form>
```

Decisiones de implementación, línea por línea:

- **<form id="add-form">:** Aunque no enviamos datos a un servidor, agrupar los controles en un <form> es semánticamente correcto: comunica que el <input> y el <button> trabajan juntos. Los lectores de pantalla lo aprovechan, y nos deja la puerta abierta a capturar el evento submit (por ejemplo, para reaccionar a la tecla Enter).
- **onsubmit="return false;":** Por defecto, enviar un formulario **recarga la página**. Eso borraría todo lo que JavaScript haya construido. Devolver false cancela ese envío. Es una medida defensiva: si el usuario presiona Enter, la página no “se reinicia”.
- **<input type="text">:** Campo de una sola línea para escribir el texto de la tarea.
- **id="add-input":** Identificador para que JavaScript pueda leer su valor.
- **placeholder="Nueva tarea":** Texto de ayuda que aparece dentro del campo cuando está vacío y desaparece al escribir. Orienta al usuario sin ocupar espacio extra.
- **autocomplete="off":** Evita que el navegador ofrezca entradas escritas anteriormente. En una app de “una sola entrada repetitiva”, esas sugerencias molestan más de lo que ayudan.
- **<button type="button">:** Es clave que sea type="button" y **no** type="submit". Un botón submit dispararía el envío del formulario; con type="button" el botón “no hace nada” hasta que JavaScript le asigne un comportamiento. Así controlamos exactamente qué pasa al hacer clic.
- **id="add-btn":** Identificador para enganchar el evento de clic desde JavaScript.

Paso 2.2: JavaScript — capturar el clic y agregar al DOM

```
const addBtn = document.getElementById('add-btn');
const input = document.getElementById('add-input');
const list = document.getElementById('task-list');

addBtn.addEventListener('click', () => {
  const texto = input.value.trim();
  if (texto === '') return; // no agregar si está vacío

  const li = document.createElement('li');
  li.textContent = texto;
  list.appendChild(li);

  input.value = ''; // limpiar campo
  input.focus(); // devolver foco para seguir escribiendo
});
```

```
});
```

Decisiones de implementación, línea por línea:

- **const addBtn = document.getElementById('add-btn');** Buscamos el botón **una sola vez** y guardamos la referencia en una constante. Usamos const porque esa referencia no va a cambiar: siempre apunta al mismo botón. Buscar el elemento una vez (en lugar de cada vez que lo necesitamos) es más eficiente y más legible.
- **const input / const list:** La misma idea para el campo de texto y la lista. Tener las tres referencias arriba, juntas, funciona como un pequeño “índice” del script.
- **addBtn.addEventListener('click', () => { ... });** Le decimos al botón “cuando ocurra un clic, ejecutá esta función”. addEventListener es la forma estándar de reaccionar a eventos. La función () => { ... } (función flecha) es el bloque que se ejecutará en cada clic.
- **const texto = input.value.trim();** input.value es lo que el usuario escribió. .trim() elimina los espacios al inicio y al final. Sin .trim(), alguien podría crear una tarea hecha solo de espacios: invisible pero presente.
- **if (texto === '') return;** Validación básica del lado del cliente. Si después de recortar no quedó nada, return corta la ejecución de la función y **no se agrega** la tarea. Nunca confiamos ciegamente en la entrada del usuario.
- **const li = document.createElement('li');** Creamos un nuevo elemento **en memoria**. Todavía no está en la página.
- **li.textContent = texto;** Ponemos el texto de la tarea dentro del . Usamos textContent (y no innerHTML) porque solo queremos texto: textContent trata el contenido como texto literal, evitando que un usuario escriba etiquetas HTML que el navegador interpretaría.
- **list.appendChild(li);** Recién acá el se inserta en el DOM, como último hijo de la lista. Hasta esta línea, el elemento existía pero era invisible.
- **input.value = '';** Vaciamos el campo para que quede listo para la próxima tarea.
- **input.focus();** Devolvemos el cursor al campo. Mejora mucho la usabilidad: el usuario puede escribir varias tareas seguidas sin volver a hacer clic en el <input>.

Por qué createElement y no innerHTML: se podría agregar la tarea con list.innerHTML += '' + texto + ''. Funciona, pero tiene dos problemas: (1) obliga al navegador a re-parsear todo el HTML de la lista, y (2) si el texto del usuario contiene < o >, se interpretaría como HTML. createElement + textContent es más seguro y más predecible.

5. Tarea 3 — Marcar tareas como completadas

Objetivo

Que cada tarea responda al clic alternando entre “activa” y “completada”, con un cambio visual claro (texto tachado y color gris).

Paso 3.1: Modificar la estructura interna del

Hasta ahora el contenía solo texto. Para la Tarea 4 vamos a necesitar también un botón de borrar **dentro del mismo **. Si dejáramos el texto suelto, no podríamos distinguir “clic en el texto” de “clic en el botón”. Por eso envolvemos el texto en su propio :

```
const li = document.createElement('li');
li.className = 'task-item';

const textSpan = document.createElement('span');
textSpan.className = 'task-text';
textSpan.textContent = texto;
li.appendChild(textSpan);
```

Decisiones de implementación, línea por línea:

- **li.className = 'task-item';** Le asignamos una clase al . Las clases sirven para aplicar estilos (CSS) y para que JavaScript identifique grupos de elementos. A diferencia de un id, una clase **se repite**: todas las tareas comparten task-item.
- **const textSpan = document.createElement('span');** Creamos un que contendrá únicamente el texto de la tarea. Un es un contenedor “neutro” en línea, ideal para envolver un fragmento de texto sin alterar el diseño.
- **textSpan.className = 'task-text';** Clase para poder estilarlo y, sobre todo, para limitar a este elemento el área que responde al clic.

- `textSpan.textContent = texto`: El texto va dentro del `<span>`, no directamente en el `<li>`.
- `li.appendChild(textSpan)`: El `<span>` queda como hijo del `<li>`.

La decisión clave: si pusiéramos el evento de clic sobre el `<li>` entero, el futuro botón de borrar (que también vive dentro del `<li>`) dispararía el toggle al hacer clic. Aislando el texto en su propio `<span>`, el área que marca/desmarca la tarea es **exactamente** el texto, y nada más.

Paso 3.2: CSS para el estado completado

```
li.task-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: #fff;
  padding: 10px 15px;
  margin-bottom: 8px;
  border-radius: 4px;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
}

li.task-item.completed .task-text {
  text-decoration: line-through;
  color: gray;
}

.task-text {
  flex-grow: 1;
  cursor: pointer;
}
```

Decisiones de implementación, línea por línea:

- `li.task-item { display: flex }`: Convertimos cada tarea en un contenedor flex. Esto nos permitirá poner el texto a la izquierda y el botón de borrar a la derecha, en la misma línea.
- `justify-content: space-between`: Empuja los hijos hacia los extremos: el texto pegado a la izquierda, el botón pegado a la derecha, con todo el espacio sobrante en el medio.
- `align-items: center`: Centra verticalmente texto y botón, para que queden alineados aunque tengan distinta altura.
- `background: #fff + box-shadow + border-radius`: Estos tres juntos producen el efecto de “tarjeta”: fondo blanco, esquinas redondeadas y una sombra sutil que la separa del fondo. La sombra usa `rgba(0,0,0,0.1)` — negro al 10 % de opacidad — para que sea apenas perceptible.
- `padding: 10px 15px`: Espacio interno (10 px arriba/abajo, 15 px a los lados) para que el texto no toque los bordes de la tarjeta.
- `margin-bottom: 8px`: Separa una tarjeta de la siguiente.
- `li.task-item.completed .task-text`: Este selector se lee así: “el `.task-text` que está dentro de un `li` que tiene **a la vez** las clases `task-item` y `completed`”. Es decir, el estilo tachado solo aplica cuando la tarea está marcada como completada.
- `text-decoration: line-through + color: gray`: El feedback visual de “tarea terminada”: texto tachado y atenuado.
- `.task-text { flex-grow: 1 }`: Le indica al texto que **crezca** para ocupar todo el espacio libre de la tarjeta. Así el botón de borrar queda empujado al extremo derecho.
- `.task-text { cursor: pointer }`: Cambia el cursor a una “manito” al pasar sobre el texto. Es una pista de usabilidad: comunica “esto es clickeable” sin necesidad de instrucciones.

Por qué la clase `completed` va en el `<li>` y no en el `<span>`: aunque el estilo afecta al texto, poner la clase en el elemento padre nos deja la libertad de, en el futuro, cambiar otros aspectos de la tarjeta completa (borde, opacidad, fondo) sin reescribir nada. La clase actúa como un “estado” del `<li>`.

Paso 3.3: JavaScript — alternar la clase

```
textSpan.addEventListener('click', () => {  
  li.classList.toggle('completed');  
});
```

Decisiones de implementación, línea por línea:

- `textSpan.addEventListener('click', ...)`: El evento se asigna al `<span>` del texto, no al `<li>`. Coherente con lo decidido en el Paso 3.1.
- `li.classList.toggle('completed')`: `classList` es la API moderna para manejar las clases de un elemento. `.toggle('completed')` **agrega** la clase si no está y la **quita** si ya está. Una sola línea reemplaza al patrón antiguo de leer `className`, buscar el texto, concatenar o recortar. Como el CSS del Paso 3.2 reacciona a esa clase, agregarla o quitarla cambia el aspecto al instante.

Nota de coherencia. En esta etapa, la interactividad (`toggle`) la reciben las tareas que crea JavaScript. Las 5 tareas predefinidas del HTML siguen siendo, por ahora, solo visuales. Esa “doble naturaleza” (unas interactivas, otras no) es incómoda a propósito: es justamente el problema que la **Tarea 7** resolverá unificando todo en un único origen de datos.

6. Tarea 4 — Botón de eliminar

Objetivo

Añadir un botón “X” a cada tarea creada por JavaScript, que permita quitarla de la lista.

Paso 4.1: Crear el botón en JavaScript

```
const deleteBtn = document.createElement('button');  
deleteBtn.className = 'delete-btn';  
deleteBtn.textContent = 'X';  
li.appendChild(deleteBtn);
```

CSS:

```
.delete-btn {  
  background: transparent;  
  border: none;  
  color: #c00;  
  font-size: 18px;  
  cursor: pointer;  
}
```

Decisiones de implementación, línea por línea:

- `document.createElement('button')`: Creamos el botón **desde JavaScript**, no en el HTML. Razón: si lo escribiéramos fijo en el HTML, solo lo tendrían las tareas predefinidas, no las nuevas. Al generarlo en el mismo lugar donde creamos cada `<li>`, garantizamos que **toda** tarea creada por JavaScript lo tenga.
- `deleteBtn.className = 'delete-btn'`: Clase para estilarlo.
- `deleteBtn.textContent = 'X'`: Usamos el carácter “X” como ícono. No necesitamos una imagen ni una librería de íconos: un carácter Unicode es suficiente y pesa cero.
- `li.appendChild(deleteBtn)`: El botón se agrega como segundo hijo del `<li>` (después del `<span>` del texto). Gracias al `display: flex` y `justify-content: space-between` del Paso 3.2, queda automáticamente a la derecha.
- `background: transparent; border: none;`: Quita la apariencia nativa de botón (fondo gris, borde). Buscamos que se vea como un **ícono de acción**, no como un botón de formulario.
- `color: #c00`: Rojo. Es la convención universal para acciones destructivas o de peligro.
- `font-size: 18px`: Agrandamos el “X” para que sea cómodo de pulsar.
- `cursor: pointer`: Otra vez, la pista visual de “esto se puede clicar”.

Paso 4.2: JavaScript — eliminar el elemento

```
deleteBtn.addEventListener('click', () => {  
  li.remove();  
});
```

Decisiones de implementación, línea por línea:

- `deleteBtn.addEventListener('click', ...)`: El evento va sobre el botón. Como el botón es un elemento aparte del `<span>` del texto, hacer clic en “X” **no** dispara el toggle. Aquí se cobra la decisión del Paso 3.1.
- `li.remove()`: Elimina el `<li>` (con todo su contenido) del DOM. Es el método moderno y directo. La alternativa antigua, `li.parentNode.removeChild(li)`, hace lo mismo pero es más larga y menos legible.

Sobre por qué asignamos un listener a cada botón. Con pocas tareas, asignar el evento directamente a cada botón es lo más simple y explícito. Existe una técnica llamada *delegación de eventos* (un solo listener en la lista que atiende los clics de todos los botones), más eficiente cuando hay cientos de elementos. Aquí priorizamos la claridad: que se vea con nitidez “este botón → esta acción”.

7. Tarea 5 — Contador de tareas

Objetivo

Mostrar arriba de la lista cuántas tareas hay en total, y que ese número se actualice solo al agregar o eliminar.

Paso 5.1: HTML del contador

```
<div id="counter">Total: 0</div>
```

Decisiones de implementación:

- **<div> con id**: Un `<div>` es un contenedor genérico, suficiente para un texto de estado. No usamos un `<p>` porque este texto no forma parte de un párrafo de lectura: es un indicador, no prosa.
- **Texto inicial Total: 0**: La página debe verse coherente incluso antes de que JavaScript actúe. Empezamos en 0.

Paso 5.2: CSS

```
#counter {  
  margin-bottom: 20px;  
}
```

Decisiones de implementación:

- **margin-bottom: 20px**: Separa el contador de la lista. La ubicación —arriba de la lista— sigue el patrón mental “resumen primero, detalle después”: el usuario ve el estado general antes de leer cada tarea.

Paso 5.3: JavaScript — centralizar la actualización en una función

```
function updateCounter() {  
  const total = document.querySelectorAll('#task-list li').length;  
  document.getElementById('counter').textContent = `Total: ${total}`;  
}
```

Y se invoca `updateCounter()` dentro de las funciones de agregar y de eliminar.

Decisiones de implementación, línea por línea:

- **function updateCounter() { ... }**: Encapsulamos la lógica de contar en una **función con nombre**. Esta es la primera vez en el proyecto que creamos una función propia, y el motivo es claro: la acción “actualizar el contador” hace falta en **dos lugares** (al agregar y al eliminar). En vez de escribir el mismo cálculo dos veces, lo escribimos una y lo llamamos por su nombre. Si mañana cambia la forma de contar, se toca **un solo lugar**.

- `document.querySelectorAll('#task-list li')`: Selecciona **todos** los `<li>` que hay dentro de `#task-list`. Devuelve una colección de elementos.
- `.length`: La cantidad de elementos de esa colección. Es decir: cuántas tareas hay en pantalla en este instante.
- `document.getElementById('counter').textContent = ...`: Escribe el resultado en el `<div>` del contador.
- ``Total: ${total}``: Una *plantilla literal* (template literal). Las comillas invertidas permiten incrustar el valor de una variable con `${...}` directamente dentro del texto, más legible que concatenar con `+`.

Idea que arrastramos. Acabamos de ver, en pequeño, el razonamiento “código repetido → extraigo una función”. En la Tarea 7 aplicaremos exactamente la misma idea, pero a un bloque de código mucho más grande: el dibujado completo de la lista.

8. Tarea 6 — División entre Activas y Completadas

Objetivo

Ampliar el contador para mostrar tres métricas: Activas: X | Completadas: Y | Total: Z. Los números deben cambiar también al marcar o desmarcar una tarea.

Paso 6.1: HTML actualizado

```
<div id="counter">Activas: 0 | Completadas: 0 | Total: 0</div>
```

Paso 6.2: JavaScript — contar por estado

```
function updateCounters() {
  const items = document.querySelectorAll('#task-list li');
  let completed = 0;
  items.forEach(li => {
    if (li.classList.contains('completed')) completed++;
  });
  const total = items.length;
  const active = total - completed;

  document.getElementById('counter').textContent =
    `Activas: ${active} | Completadas: ${completed} | Total: ${total}`;
}
```

Decisiones de implementación, línea por línea:

- **function updateCounters()**: Renombramos la función de `updateCounter` (singular) a `updateCounters` (plural). El nombre **comunica intención**: ahora calcula varias métricas, no una. Mantener nombres honestos es parte de escribir código legible.
- **const items = document.querySelectorAll('#task-list li')**: Tomamos todos los `<li>` una sola vez y los guardamos, porque vamos a usarlos dos veces (para contar completadas y para el total).
- **let completed = 0**: Un contador que **va a cambiar**, por eso `let` y no `const`. Empieza en 0.
- **items.forEach(li => { ... })**: Recorremos cada `<li>` de la colección.
- **if (li.classList.contains('completed')) completed++**: `classList.contains(...)` pregunta si el elemento tiene esa clase. Si la tiene, está completada y sumamos 1. Reutilizamos la clase `completed` —que ya existía para el estilo del Paso 3.2— como **indicador de estado**. No necesitamos un atributo extra.
- **const total = items.length**: El total es la cantidad de `<li>`.
- **const active = total - completed**: Las activas son, por definición, las que no están completadas. Calcularlo por resta evita recorrer la lista una segunda vez.
- **Asignación final**: Escribimos las tres métricas en el contador con una plantilla literal.

Punto fácil de olvidar. Hay que llamar a `updateCounters()` **también dentro del toggle**: al marcar una tarea como completada, el reparto Activas/Completadas cambia aunque el total no. Esto obliga a pensar de forma sistemática: “¿qué eventos afectan a qué partes de la vista?”. Marcar como completada afecta al contador → entonces el toggle debe refrescar el contador.

9. Tarea 7 — Persistencia y la función de renderizado

Esta es la tarea más larga y la más importante de la cursada. Vamos a hacer dos cosas grandes: (1) lograr que las tareas **sobrevivan a recargas** usando `localStorage`, y (2) reorganizar el dibujado de la lista. Para lo segundo seguiremos una secuencia deliberada: **primero escribiremos el código de dibujado “suelto”, lo veremos funcionar, descubriremos que se repite, y recién entonces lo encapsularemos en una función `renderTasks()`**. Tomate tu tiempo con esta tarea.

Objetivo

Que las tareas persistan entre recargas de página, que exista un botón para borrar todo, y —en el camino— aprender por qué conviene tener **una sola función** encargada de dibujar la lista.

Paso 9.1: El cambio de mentalidad — ¿dónde vive la verdad?

Hasta la Tarea 6, la “fuente de verdad” de la aplicación eran **los elementos del DOM**. Para saber cuántas tareas había, contábamos `<li>`. Para saber si una estaba completada, mirábamos si su `<li>` tenía la clase `completed`. Funcionaba, pero tiene un defecto fatal: **si el usuario recarga la página, el DOM se reconstruye desde cero y todo se pierde**.

La decisión de esta tarea: **la fuente de verdad pasa a ser un arreglo de objetos JavaScript**. El DOM deja de “ser” los datos y pasa a ser una **vista** de esos datos —un reflejo—. Y `localStorage` actúa como el “disco duro” donde ese arreglo se guarda entre sesiones.

Cada tarea se representará así:

```
{ text: 'Comprar comida', completed: false }
```

Es un objeto con dos propiedades: el texto y un booleano que indica si está completada. El arreglo completo es, simplemente, una lista de estos objetos.

Consecuencia en el HTML. Como las tareas las va a dibujar JavaScript a partir del arreglo, **quitamos del HTML los 5 `<li>` predefinidos**. La lista queda vacía:

```
<ul id="task-list"></ul>
```

Las tareas iniciales ya no son HTML: serán objetos dentro del arreglo (lo veremos en el Paso 9.9).

Paso 9.2: Funciones de persistencia (leer y escribir en `localStorage`)

```
const STORAGE_KEY = 'task_manager_tasks';

function loadTasks() {
  const data = localStorage.getItem(STORAGE_KEY);
  return data ? JSON.parse(data) : [];
}

function saveTasks(tasks) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}
```

Decisiones de implementación, línea por línea:

- **`const STORAGE_KEY = 'task_manager_tasks'`**: `localStorage` funciona como un diccionario de pares clave→valor. Guardamos el nombre de **nuestra** clave en una constante. Ventaja: si escribimos mal la clave en algún lado, el error es inmediato; y si algún día queremos renombrarla, se cambia en un único punto. Escribir `'task_manager_tasks'` a mano en cada uso sería frágil.
- **`function loadTasks()`**: Función que **devuelve** el arreglo de tareas guardado.
 - **`localStorage.getItem(STORAGE_KEY)`**: Lee el valor almacenado bajo nuestra clave. Si nunca se guardó nada, devuelve `null`.

- **return data ? JSON.parse(data) : []**: Un operador ternario. Se lee: “¿data tiene contenido? Entonces convértelo de texto a arreglo con JSON.parse. Si no (primera visita, data es null), devolvé un arreglo vacío []”. Ese **valor de respaldo** [] es importante: si devolviéramos null, la próxima línea que intente hacer .forEach() sobre el resultado fallaría.
- **function saveTasks(tasks)**: Recibe el arreglo y lo **escribe** en localStorage.
 - **JSON.stringify(tasks)**: localStorage **solo puede almacenar texto**, no objetos ni arreglos. JSON.stringify convierte nuestro arreglo de objetos en una cadena de texto con formato JSON.
 - **localStorage.setItem(STORAGE_KEY, ...)**: Guarda esa cadena bajo nuestra clave.

loadTasks y saveTasks son una pareja simétrica: una traduce de “disco” a memoria (JSON.parse), la otra de memoria a “disco” (JSON.stringify).

Paso 9.3: Primera versión — dibujar la lista con código suelto (sin función)

Ya tenemos cómo **guardar y leer** el arreglo. Ahora necesitamos **mostrarlo** en pantalla. Empecemos de la forma más directa posible: escribimos el código de dibujado tal cual, “suelto”, al final del script. El único objetivo de este paso es **ver la lista en la página**.

```
// --- Dibujar la lista a partir del arreglo (primera versión, sin función) ---
const list = document.getElementById('task-list');
const tasks = loadTasks();

list.innerHTML = ''; // limpiar lo que hubiera antes

tasks.forEach((task, index) => {
  const li = document.createElement('li');
  li.className = 'task-item' + (task.completed ? ' completed' : '');
  li.dataset.index = index;

  const textSpan = document.createElement('span');
  textSpan.className = 'task-text';
  textSpan.textContent = task.text;
  li.appendChild(textSpan);

  const deleteBtn = document.createElement('button');
  deleteBtn.className = 'delete-btn';
  deleteBtn.textContent = 'X';
  li.appendChild(deleteBtn);

  list.appendChild(li);
});
```

Decisiones de implementación, línea por línea:

- **const list = document.getElementById('task-list')**: Referencia a la vacía.
- **const tasks = loadTasks()**: Traemos el arreglo desde localStorage. En la primera visita será [].
- **list.innerHTML = ''**: Vacía la lista antes de dibujar. Aquí no hace falta (la ya está vacía), pero lo incluimos a propósito: cuando volvamos a dibujar tras un cambio, este “borrón y cuenta nueva” será imprescindible.
- **tasks.forEach((task, index) => { ... })**: Recorremos el arreglo. forEach nos entrega, en cada vuelta, **dos** datos: el objeto task y su posición numérica index.
- **document.createElement('li')**: Creamos el de esta tarea.
- **li.className = 'task-item' + (task.completed ? ' completed' : '')**: Asignamos la clase. Si la tarea está completada, **concatenamos** también la clase completed (con un espacio inicial). Así, una tarea completada se dibuja ya tachada, sin pasos extra. El estado del objeto **determina** el aspecto del DOM.
- **li.dataset.index = index**: Guardamos en el la posición que ocupa la tarea en el arreglo. dataset.index crea el atributo data-index en el HTML. Esto será nuestro “puente”: cuando el usuario haga clic en una tarea, sabremos **qué posición del arreglo** modificar.
- **const textSpan ...**: Igual que en la Tarea 3 — el texto va en su propio , leyendo task.text del objeto.
- **const deleteBtn ...**: Igual que en la Tarea 4 — el botón “X”.
- **list.appendChild(li)**: Insertamos el terminado en la lista.

Si recargás la página ahora, **vas a ver la lista dibujada**. Pero todavía le faltan dos cosas:

1. No tiene eventos: hacer clic en una tarea o en “✕” no hace nada.
2. Solo se dibuja **una vez**, al cargar la página.

Esos dos faltantes nos llevan al siguiente paso.

Paso 9.4: El problema — necesitamos volver a dibujar tras cada cambio

Pensemos qué pasa cuando el usuario **agrega** una tarea. La operación, en términos del nuevo modelo, es:

1. Leer el arreglo (`loadTasks()`).
2. Agregarle la tarea nueva (`push`).
3. Guardar el arreglo (`saveTasks()`).
4. **Volver a dibujar la lista**, para que la pantalla refleje el cambio.

El paso 4 es el problema. Escrita “a mano”, la función de agregar quedaría así:

```
// addTask — versión que repite el código de dibujado (NO es la versión final)
function addTask(text) {
  if (!text.trim()) return;

  const tasks = loadTasks();
  tasks.push({ text: text.trim(), completed: false });
  saveTasks(tasks);

  // ...y ahora hay que volver a dibujar TODO:
  const list = document.getElementById('task-list');
  list.innerHTML = '';
  tasks.forEach((task, index) => {
    const li = document.createElement('li');
    li.className = 'task-item' + (task.completed ? ' completed' : '');
    li.dataset.index = index;

    const textSpan = document.createElement('span');
    textSpan.className = 'task-text';
    textSpan.textContent = task.text;
    li.appendChild(textSpan);

    const deleteBtn = document.createElement('button');
    deleteBtn.className = 'delete-btn';
    deleteBtn.textContent = '✕';
    li.appendChild(deleteBtn);

    list.appendChild(li);
  });
}
```

Y ahora pensemos en **eliminar** una tarea, y en **marcarla como completada**. Cada una de esas operaciones también necesita guardar y **volver a dibujar la lista entera**. Es decir: ese mismo bloque del `forEach` —unas 15 líneas— tendría que aparecer **copiado y pegado en tres o cuatro funciones distintas**.

Eso es un problema serio:

- **Repetición:** el mismo código en cuatro lugares.
- **Mantenimiento:** si mañana queremos agregar, por ejemplo, una fecha a cada tarea, hay que recordar tocar **los cuatro** bloques. Olvidar uno produce un bug silencioso.
- **Legibilidad:** las funciones `addTask`, `deleteTask`, etc. se vuelven larguísimas y cuesta ver qué hace cada una de verdad.

Este es exactamente el mismo razonamiento que aplicamos en pequeño en la Tarea 5 (`updateCounter`). La regla es:

Cuando un mismo bloque de código se necesita en varios lugares, se extrae a una función con nombre y se lo llama por ese nombre.

Paso 9.5: La decisión — encapsular el dibujado en renderTasks()

Tomamos el bloque de dibujado del Paso 9.3 y lo **encerramos dentro de una función**. La función recibe el arreglo como parámetro y se encarga de **todo** lo necesario para que la pantalla refleje ese arreglo: limpiar, recorrer, crear los elementos, conectar los eventos y actualizar el contador.

```
function renderTasks(tasks) {
  const list = document.getElementById('task-list');
  list.innerHTML = ''; // limpiar la vista anterior

  tasks.forEach((task, index) => {
    const li = document.createElement('li');
    li.className = 'task-item' + (task.completed ? ' completed' : '');
    li.dataset.index = index;

    const textSpan = document.createElement('span');
    textSpan.className = 'task-text';
    textSpan.textContent = task.text;
    li.appendChild(textSpan);

    const deleteBtn = document.createElement('button');
    deleteBtn.className = 'delete-btn';
    deleteBtn.textContent = 'X';
    li.appendChild(deleteBtn);

    // Eventos: ahora SÍ los conectamos, dentro del render
    textSpan.addEventListener('click', () => {
      toggleComplete(index);
    });
    deleteBtn.addEventListener('click', () => {
      deleteTask(index);
    });

    list.appendChild(li);
  });

  updateCounters(tasks); // contador, al final del render
}
```

Qué cambió respecto del código suelto del Paso 9.3, y por qué:

- **function renderTasks(tasks) { ... }:** El bloque ahora vive dentro de una función con nombre. El nombre renderTasks (“renderizar las tareas”) **documenta su propósito**: cualquiera que lea el código entiende, sin comentarios, qué hace.
- **El parámetro tasks:** La función no sale a buscar el arreglo por su cuenta; lo **recibe**. Esto la hace reusable: quien la llama decide qué arreglo dibujar. También la hace más fácil de probar y de razonar — “dame un arreglo y te dibujo exactamente eso”.
- **Los addEventListener se mudaron adentro:** Cada vez que renderTasks reconstruye un , le conecta sus eventos. Esto es **necesario**: como en cada render destruimos los viejos (innerHTML = '') y creamos otros nuevos, los eventos que tenían los viejos se pierden con ellos. Los nuevos necesitan sus propios eventos.
- **textSpan llama a toggleComplete(index) y deleteBtn llama a deleteTask(index):** los eventos ya no manipulan el DOM directamente (como en las Tareas 3 y 4). Ahora avisan a una función que **modifica el arreglo**. Le pasan el index para que esa función sepa **cuál** tarea tocar. Definiremos toggleComplete y deleteTask en el paso siguiente.
- **updateCounters(tasks) al final:** el contador se calcula como **parte** del render. Volveremos sobre esto en el Paso 9.8.

Lo que ganamos con esta única función:

1. **Una sola “puerta” para dibujar.** Nadie más toca la . Si la pantalla está mal, el problema está en renderTasks y en ningún otro lado.
2. **Las operaciones se vuelven triviales.** Como veremos enseguida, agregar, borrar o completar se reducen a “modifico el arreglo, guardo, y llamo a renderTasks”.
3. **Un solo lugar para cambiar el dibujado.** ¿Querés agregar una fecha a cada tarjeta? Se modifica renderTasks y listo: todas las tareas, vengan de donde vengan, se dibujan igual.

Sobre el orden de las funciones. Dentro de `renderTasks` llamamos a `toggleComplete`, `deleteTask` y `updateCounters`, que definiremos después en el archivo. JavaScript lo permite: las **declaraciones de función** (`function nombre() {}`) se “elevan” (hoisting) y están disponibles en todo el script. Lo que importa es que esas funciones existan **en el momento en que `renderTasks` se ejecuta**, no antes.

Paso 9.6: Las operaciones — agregar, eliminar, completar

Con `renderTasks` resuelto, las tres operaciones quedan cortas y casi idénticas entre sí. Todas siguen el mismo ciclo: **leer → modificar → guardar → renderizar**.

```
function addTask(text) {
  if (!text.trim()) return;
  const tasks = loadTasks();
  tasks.push({ text: text.trim(), completed: false });
  saveTasks(tasks);
  renderTasks(tasks);
  document.getElementById('add-input').value = '';
}

function deleteTask(index) {
  const tasks = loadTasks();
  tasks.splice(index, 1);
  saveTasks(tasks);
  renderTasks(tasks);
}

function toggleComplete(index) {
  const tasks = loadTasks();
  tasks[index].completed = !tasks[index].completed;
  saveTasks(tasks);
  renderTasks(tasks);
}
```

Decisiones de implementación, línea por línea:

- **`addTask(text)`** — agregar:
 - **`if (!text.trim()) return`**: la misma validación de la Tarea 2 — no se agregan tareas vacías.
 - **`const tasks = loadTasks()`**: **siempre** partimos de leer el arreglo guardado. Nunca trabajamos con una copia “vieja” que tengamos en memoria; así no perdemos cambios hechos en otra pestaña o en otra sesión.
 - **`tasks.push({ text: text.trim(), completed: false })`**: agregamos un objeto nuevo al final del arreglo. Nace con `completed: false` (toda tarea empieza activa).
 - **`saveTasks(tasks)`**: persistimos el arreglo modificado.
 - **`renderTasks(tasks)`**: redibujamos. Una sola línea reemplaza a las 15 que copiábamos en el Paso 9.4.
 - **`document.getElementById('add-input').value = ''`**: limpiamos el campo de texto.
- **`deleteTask(index)`** — eliminar:
 - **`tasks.splice(index, 1)`**: `splice(posición, cantidad)` borra elementos del arreglo. Acá borra **1** elemento en la posición `index`. El arreglo se “compacta” solo: no quedan huecos.
- **`toggleComplete(index)`** — completar/descompletar:
 - **`tasks[index].completed = !tasks[index].completed`**: accedemos a la tarea en la posición `index` e **invertimos** su propiedad `completed` con el operador `!` (negación). Si era `false` pasa a `true` y viceversa.

El gran cambio respecto de las Tareas 3 y 4. Antes, el clic modificaba el DOM directamente (`li.classList.toggle`, `li.remove`). Ahora **ninguna operación toca el DOM**. Todas modifican el **arreglo**, lo guardan, y dejan que `renderTasks` reconstruya la vista. El DOM siempre es un reflejo fiel del arreglo. Esto elimina de raíz una clase entera de errores: que la pantalla “diga” algo distinto de lo que indican los datos.

Paso 9.7: ¿Por qué `innerHTML = ''` y redibujar todo?

`renderTasks` borra la lista completa y la reconstruye desde cero en cada llamada. A primera vista parece un derroche: si el usuario borró una sola tarea, ¿por qué redibujar las otras cuatro?

La respuesta es **simplicidad y seguridad**:

- Editar el DOM “quirúrgicamente” (encontrar el `<li>` exacto, quitarlo, y además reajustar el `data-index` de todos los `<li>` siguientes) es complejo y propenso a errores de sincronización.
- Reconstruir todo a partir del arreglo es **imposible de desincronizar**: la pantalla, por definición, siempre coincide con los datos.
- Para una lista de 5, 50 o incluso unos cientos de ítems, el costo es **imperceptible** para el usuario.

A este enfoque se lo llama **re-render completo**. Es la decisión correcta para un proyecto introductorio. Si la lista creciera a miles de elementos, recién ahí valdría la pena una técnica de actualización parcial (lo que los frameworks modernos llaman *diffing*).

Paso 9.8: Refactor del contador — `updateCounters` lee del arreglo

En la Tarea 6, `updateCounters` contaba **nodos del DOM** (`<li>` con la clase `completed`). Ahora que la fuente de verdad es el arreglo, ese enfoque quedó **incoherente**: estaríamos preguntándole a la “vista” algo que el “modelo” sabe mejor. Lo corregimos para que cuente sobre el arreglo:

```
function updateCounters(tasks) {
  const total = tasks.length;
  const completed = tasks.filter(t => t.completed).length;
  const active = total - completed;
  const counterDiv = document.getElementById('counter');
  counterDiv.textContent = `Activas: ${active} | Completadas: ${completed} | Total: ${total}`;
}
```

Decisiones de implementación, línea por línea:

- **function updateCounters(tasks)**: ahora recibe el arreglo como parámetro, igual que `renderTasks`. Coherente con el nuevo modelo.
- **const total = tasks.length**: el total es, simplemente, la cantidad de objetos del arreglo.
- **const completed = tasks.filter(t => t.completed).length**: `filter` arma un **nuevo arreglo** solo con las tareas cuyo `completed` es `true`; su `.length` es la cantidad de completadas. Es más declarativo que recorrer con un `if` y un contador manual.
- **const active = total - completed**: las activas, por resta.
- **Asignación final**: escribimos las tres métricas en el contador.

Otra ventaja de centralizar el dibujado. Fijate que `updateCounters` se llama **una sola vez**, al final de `renderTasks`. Ya no necesitamos acordarnos de invocarla en cada operación (como advertíamos en la Tarea 6): como **toda** operación termina llamando a `renderTasks`, y `renderTasks` llama a `updateCounters`, el contador se actualiza solo, siempre. Un olvido menos posible.

Paso 9.9: Tareas predefinidas (solo en la primera visita)

```
const initialTasks = [
  { text: 'Comprar comida', completed: false },
  { text: 'Pasear al perro', completed: false },
  { text: 'Estudiar JavaScript', completed: false },
  { text: 'Lavar la ropa', completed: false },
  { text: 'Llamar a mamá', completed: false }
];

if (!localStorage.getItem(STORAGE_KEY)) {
  saveTasks(initialTasks);
}
```

Decisiones de implementación, línea por línea:

- **const initialTasks = [...]**: las 5 tareas de ejemplo, ahora como **objetos** dentro de un arreglo —no como `<li>` en el HTML—. Recordá que en el Paso 9.1 vaciamos la `<ul>`.
- **if (!localStorage.getItem(STORAGE_KEY))**: la condición se lee “si **no** hay nada guardado bajo nuestra clave”. Es decir: solo en la **primera visita**.

- **saveTasks(initialTasks)**: en esa primera visita, sembramos el `localStorage` con las tareas de ejemplo. En visitas posteriores la condición es falsa y **respetamos los datos del usuario**: si ya creó, borró o completó tareas, no las pisamos.

Por qué objetos y no HTML. Si las tareas iniciales fueran `<li>` en el HTML, tendrían un origen distinto al de las tareas nuevas, y `renderTasks` no las “conocería”. Al definir las como objetos del arreglo, pasan exactamente por el mismo `renderTasks` que cualquier otra. **Un único camino para todas las tareas.**

Paso 9.10: Botón “Borrar todo”

HTML:

```
<button id="clear-all">Borrar todo</button>
```

CSS:

```
#clear-all {
  margin-top: 10px;
  padding: 6px 10px;
  font-size: 0.9em;
  background: #dc3545;
  color: #fff;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
```

JavaScript:

```
function clearAll() {
  localStorage.removeItem(STORAGE_KEY);
  renderTasks([]);
}

document.getElementById('clear-all').addEventListener('click', () => {
  if (confirm('¿Seguro que quieres borrar todas las tareas?')) {
    clearAll();
  }
});
```

Decisiones de implementación, línea por línea:

- **localStorage.removeItem(STORAGE_KEY)**: borra del almacenamiento persistente todo lo guardado bajo nuestra clave.
- **renderTasks([])**: redibuja con un **arreglo vacío**. Resultado: la lista queda vacía y el contador en `0 | 0 | 0`. Las dos acciones —limpiar el “disco” y limpiar la “vista”— deben ir juntas para que datos y pantalla sigan coincidiendo.
- **confirm('¿Seguro...?')**: “Borrar todo” es una acción **destructiva e irreversible**. `confirm` abre un diálogo Sí/No del navegador: solo si el usuario acepta, se ejecuta `clearAll()`. Es una salvaguarda de usabilidad imprescindible.
- **background: #dc3545** (rojo): mismo código de color “peligro” del botón de eliminar individual. La coherencia visual ayuda al usuario a reconocer “esto borra cosas” sin leer.

Paso 9.11: Inicialización — dibujar al cargar la página

```
renderTasks(loadTasks());
```

Decisiones de implementación:

- **Una sola línea, al final del script.** Lee el arreglo desde `localStorage` (`loadTasks()`) y se lo pasa a `renderTasks()` para el primer dibujado. Es lo que hace que, al abrir o recargar la página, veas tus tareas.
- **No hace falta DOMContentLoaded.** Esa envoltura sirve para esperar a que el DOM esté listo. Pero como el `<script>` está al final del `<body>` (Paso 1.1), cuando esta línea se ejecuta **el HTML ya está completamente parseado**. La envoltura sería redundante.

Paso 9.12: Conexión del botón “Añadir”

Para cerrar, reconectamos el botón de agregar al nuevo `addTask`:

```
document.getElementById('add-btn').addEventListener('click', () => {  
  const input = document.getElementById('add-input');  
  addTask(input.value);  
});
```

Decisiones de implementación:

- El listener lee el valor del campo y se lo pasa a `addTask`. Toda la lógica (validar, modificar el arreglo, guardar, redibujar, limpiar el campo) ya vive dentro de `addTask`. El listener solo **conecta** un evento de la interfaz con una operación del modelo. Esa división —“la interfaz avisa, el modelo decide”— es la idea que ordena todo el `script.js` final.

10. Arquitectura final: decisiones clave

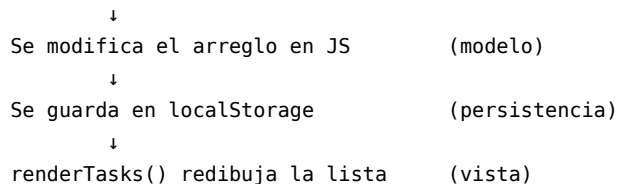
10.1 Separación de responsabilidades

Capa	Rol	Archivo
Estructura	Define qué elementos existen	<code>index.html</code>
Presentación	Define cómo se ven y se comportan visualmente	<code>estilo.css</code>
Conducta y datos	Maneja la lógica, el estado y la persistencia	<code>script.js</code>

10.2 Modelo y Vista: un flujo en una sola dirección

En lugar de tocar el DOM “aquí y allá”, el proyecto adopta un flujo **unidireccional**:

El usuario interactúa



Ventajas, pedagógicas y técnicas:

- **Predecibilidad:** siempre sabés dónde está la verdad — en el arreglo.
- **Depuración fácil:** un `console.log(loadTasks())` en cualquier momento te muestra el estado real de la app.
- **Extensibilidad:** si mañana querés agregar “fecha de vencimiento” o “prioridad”, agregás una propiedad al objeto y un par de líneas a `renderTasks`. Nada más.

10.3 Por qué `renderTasks` es una sola función

Esta fue la refactorización central de la Tarea 7. La progresión fue: **código suelto → repetición detectada → función extraída**. Tener una única función de dibujado significa que:

- Existe **una sola puerta** hacia el DOM de la lista. Ningún otro código toca la `<ul>`.
- Las operaciones (`addTask`, `deleteTask`, `toggleComplete`, `clearAll`) son cortas: modifican el arreglo y delegan el dibujado.
- Cambiar el aspecto de las tareas se hace en **un único lugar**.

Es la misma idea que aplicamos en miniatura en la Tarea 5 con `updateCounter`: **código repetido → función con nombre**.

10.4 Identificar tareas con `dataset.index`

Usamos el **índice numérico** del arreglo (`data-index`) para saber qué tarea tocar al hacer clic. Es la solución más simple para un ejercicio introductorio. Tiene un límite conocido: al borrar una tarea del medio, los índices de las siguientes se desplazan, y por eso

debemos redibujar después de cada cambio (lo cual ya hacemos). En una aplicación real con edición concurrente se preferiría un **ID único** por tarea (por ejemplo `crypto.randomUUID()`), independiente de la posición.

10.5 Re-render completo vs. actualización parcial

Decidimos borrar y reconstruir la lista entera (`innerHTML = '' + forEach`). Para 5–50 tareas es instantáneo y mucho menos propenso a errores que editar nodos uno por uno. Si la lista creciera a cientos o miles de ítems, recién entonces convendría una actualización quirúrgica (*diffing*).

11. Checklist de aprendizaje

Al finalizar esta guía, el alumno debería ser capaz de explicar y reproducir:

- ☐ Estructura semántica de un documento HTML5 con `lang`, `charset` y separación de archivos.
 - ☐ Diferencia entre `id` (elemento único) y `class` (estilo o estado reutilizable).
 - ☐ Uso de Flexbox para centrar y distribuir contenido.
 - ☐ Captura de eventos con `addEventListener`.
 - ☐ Creación dinámica de elementos con `document.createElement` y por qué se prefiere a `innerHTML` para insertar texto.
 - ☐ Alternancia de clases CSS con `classList.toggle` y consulta con `classList.contains`.
 - ☐ Razonamiento “código repetido → extraer una función con nombre” (visto en `updateCounter` y, en grande, en `renderTasks`).
 - ☐ Persistencia en el navegador con `localStorage`, `JSON.stringify` y `JSON.parse`.
 - ☐ Diferencia entre **modelo** (arreglo de objetos, la verdad) y **vista** (el DOM, un reflejo).
 - ☐ El ciclo **leer → modificar → guardar → renderizar** y por qué ninguna operación toca el DOM directamente.
 - ☐ Por qué conviene una única función `renderTasks` como “puerta” hacia el DOM de la lista.
-

Apéndice: estructura de archivos final

```
proyecto-tasks/  
├─ index.html      → Estructura y controles de la interfaz  
├─ estilo.css      → Reglas de presentación y estados visuales  
├─ script.js       → Lógica, modelo de datos, eventos y persistencia  
└─ GUIA-GESTOR-TAREAS.md → Este documento
```

Consejo para practicar. Replicá el proyecto desde cero con esta guía cerrada, y abrila solo cuando te bloquee. La repetición activa —“hacer sin mirar”— es la que consolida el aprendizaje. Y un ejercicio extra muy recomendable: una vez que tu versión funcione, intentá explicarle en voz alta a un compañero **por qué `renderTasks` es una sola función**. Si podés explicarlo, lo entendiste.